RX / SPORTS ANALYTICS

PPP Implementation Guide

NFL and NBA models
From source observations to market probabilities

LOOKBACK / NEW SEASONS / CODE LOGIC / PRICING

A guide to the implementation in this repository, including the exact history windows, minimum samples, seasonal carryover, chronological calibration, saved forecast behavior and limitations.

Read section 0 first for the answers to how many games are used and what happens when a new season has little or no history.

The original research PDF is retained separately as a source reference. This manual explains what the code actually implements; advanced proposals in the source papers are not represented as completed features.

Code review date: 20 October 2026
Documentation source: Models/Game_Outcome/PPP/README.md

Contents

0. Lookback, new seasons and insufficient history **3**

0.1 How many games do we look back? . . . 3

0.2 Recency weighting is not a season reset . . . 3

0.3 What happens at the start of a new season? . . . 4

0.4 Why a large input file can still fail . . . 4

0.5 Cutoffs, refreshes and frozen estimates . . . 5

1. Files, sources and execution **5**

2. How the implementation maps to the PDFs **6**

3. NFL data contract and scoring definitions **6**

4. NFL efficiency estimation **7**

4.1 Intuitive baseline and paper example . . . 7

4.2 Default jointly fitted model . . . 7

5. NFL volume and score means **8**

6. Chronological calibration and uncertainty **8**

7. Settlement, market probabilities, fair odds and Edge **9**

8. NBA-specific PPP implementation **9**

8.1 Inputs and possession estimate . . . 9

8.2 League rates, shrinkage and matchup . . . 9

8.3 NBA validation and spread settlement . . . 10

9. Configuration and saved-market integration **10**

10. Verification and practical limits **11**

This folder contains two operational pregame models. **NFL PPP** means points per offensive play; **NBA PPP** means points per possession. Their efficiencies, pace estimates and calibration samples are fitted separately. They share the market-pricing interface, not fitted team ratings or football assumptions.

Implementation PDF: PPP Implementation Guide. This guide documents the checked-in code. The separately named strategy PDF below remains the original research reference, not the implementation manual.

0. Lookback, new seasons and insufficient history

0.1 How many games do we look back?

The live saved-market service requests **the current sport season and the immediately previous season**, determined from the saved snapshot's cutoff, not the computer's current date. It does not simply take the last four NFL games or the last ten NBA games. Those numbers are minimum team requirements, not rolling windows. Every valid, available game in the requested seasons can enter the final ratings fit; old games receive less weight. Regular season and postseason are included. No preseason observations are used.

Rule	NFL PPP	NBA PPP
Live season index S	Cutoff year minus 1 in January/February; otherwise cutoff year	Cutoff year minus 1 in January-September; otherwise cutoff year
Seasons requested	S-1 and S	S-1 and S; year is the starting year of the NBA season
Hard last-N rating window	None within the eligible requested seasons	None within the eligible requested seasons
Recency half-life	180 calendar days	90 calendar days
Minimum league training games	50 distinct games	100 distinct games
Minimum selected-team history	4 earlier appearances for each team	10 earlier appearances for each team
Error sample used for uncertainty	Latest 250 eligible held-out games; at least 40	Latest 300 eligible held-out games; at least 100
Calibration holdout unit	Entire season/week	Entire UTC game date
Availability assumption	Game's UTC date +36 hours	Game's UTC date +2 days

League counts are distinct games, not team-game rows. One complete game supplies two team rows but only one paired score error. Team counts can span both seasons; there is no separate minimum for games in the new season. Rating history and the smaller score-error calibration sample are different datasets.

0.2 Recency weighting is not a season reset

The raw weight is `2^(-age_in_calendar_days / half_life_days)`. NFL weights for ages 0, 180 and 360 days are 1, 0.5 and 0.25. NBA weights for ages 0, 90 and 180 days are 1, 0.5 and 0.25. An offseason counts as elapsed time. A season boundary does not automatically erase or zero the preceding season's games, and there is no explicit fixed 70/30 old-season/new-season blend.

For example, an NFL game 240 days old has weight about 0.397; eight such games have about 3.17 units of raw weight before any other exposures or penalties. Four recent games near weight 1 can therefore have comparable raw game weight. This is only a weighting illustration: NFL ratings also use play exposure and ridge penalties, so the actual rating is not a plain weighted score average.

NBA PPP first computes league rates with raw weights, then divides all team-fit weights by the newest eligible league observation's weight. This preserves relative ages while keeping its ten-equivalent-game league prior from dominating solely because all history aged during the offseason. The normalization is to the newest observation in the league cohort, not each team's newest game. NFL PPP does not apply that normalization; its fixed ridge/prior strengths can therefore matter more when all raw observations are old.

0.3 What happens at the start of a new season?

The following are examples of selection rules, not live forecasts:

Scenario	Requested history and result
NFL Week 1 in September 2026, zero 2026 games	Request 2025 and 2026. If 2026 has no completed usable data, use available 2025 records with a source notice. A forecast can still succeed if team, league and calibration requirements pass.
NFL Week 3 in 2026, two earlier new-season games per team	Keep eligible 2025 history and the available 2026 games. Four earlier appearances means four combined across the cohort, not four in 2026.
NBA opening night in October 2026	Season 2026 means 2026-27. Request 2025-26 and 2026-27. Prior-season observations can satisfy the ten-appearance team threshold without any new-season games.
NBA after three completed games per team	Add games whose availability lag has passed to the previous season's history. Three new games do not replace all prior-season games.
NFL game in January 2027	The service still regards 2026 as S, requesting 2025/2026; it does not mistake the postseason for the next NFL season.
Either team's total eligible history below its minimum	No probability estimate is returned. Saved market prices remain available; there is no fabricated league-average forecast for that team.

Current-season missing data is a source-availability condition, not a special model switch. The data builders try requested sources, preserve successful seasons, and report unavailable inputs. Prior-season carryover is used only when it actually exists and passes validation. The app does **not** automatically reach back to S-2 if S-1/S fail to provide enough history, and it does not silently reduce minimum counts or insert sample data.

Team continuity is by canonical franchise abbreviation, not by roster. A major offseason trade, new quarterback, coaching change or injury does not reset old observations or create an automatic adjustment. Historical carryover helps with sample size but can be stale after structural changes; these models do not yet measure those changes explicitly. Ten old NBA appearances or four NFL appearances are eligibility checks, not proof that the sample represents the current team.

0.4 Why a large input file can still fail

At each historical holdout, the model needs enough earlier league games and earlier appearances for both teams. Early games in a supplied dataset often cannot produce error pairs. A file with exactly 50 NFL games or exactly 100 NBA games therefore does not guarantee the required 40 or 100 calibration errors. Availability lags, missing pairs and invalid sources further reduce eligible history. The final estimate fails if there are too few chronological errors or if their score covariance produces nonpositive spread/total variance.

The ordinary two-season app window is fixed; it is not an adaptive search for enough history. For a research run, CLI `--seasons` can explicitly provide a longer history, for example `--seasons 2024 2025 2026`. That expands the standalone fit only; it does not change the live service's two-season selection. With `--as-of`, also specify `--seasons` explicitly for historical runs: CLI default seasons follow the wall clock, whereas the live service derives seasons from the snapshot cutoff.

0.5 Cutoffs, refreshes and frozen estimates

The live cutoff is `min(snapshot pulled_at, kickoff/tipoff)` in UTC. A post-start quote still receives a pregame estimate, not an in-game forecast. An observation must be available strictly before that cutoff and dated before its UTC calendar date. Same-day games and games still inside the conservative availability lag are excluded even if a source already lists a final score.

Frozen forecast identity includes model version, configuration, teams, cutoff and neutral status. Reopening the same snapshot reuses its estimate. Source hashes are recorded as provenance; they are not part of the cache key, so merely refreshing source files does not automatically refit an already frozen identity. A changed configuration/version or different cutoff produces a new identity. Public source refreshes use no Odds API credits; only the explicit Pull Live Markets action downloads new betting quotes and can consume credits. Raw source revisions can affect a newly fitted historical estimate: cutoff filtering is not a reconstruction of the exact source revision known at the old date.

1. Files, sources and execution

File	Responsibility
PPP.py	NFL ratings, opportunity forecasts, chronological calibration, spread/total settlement and common American-odds functions.
NBA_PPP.py	NBA box-score possession baseline, shrunk efficiency/pace, date-based calibration and no-tied-final-score spread settlement.
__init__.py	Public NFL exports; existing `from Models.Game_Outcome.PPP import PPPModel` imports remain valid.
__main__.py	Package launcher for NFL PPP.

The primary NFL reference is the included PPP research PDF, particularly sections 1-5. Its SHA-256 is `103fbec0dcceb15df5656ef72f8d55fc117b60959626303ab12b765b26c316f8`. The NBA method comes from sections 6.2-6.3 and 7 of the included EPA research PDF. Page references below use the printed section labels; the EPA PDF has cover/contents pages before its numbered body. The source PDFs are unchanged copies.

Run commands from the project root with the project's Python environment:

```
# NFL: prepare public observations.
python -m Models.Game_Outcome.PPP --seasons 2025 2026

# NFL: prepare, fit and write a standalone forecast.
python -m Models.Game_Outcome.PPP --seasons 2025 2026 --home SF --away WSH --as-of
    2026-10-19T23:35:23+00:00

# NBA: season 2025 means 2025-26.
python -m Models.Game_Outcome.PPP.NBA_PPP --seasons 2025 --home GSW --away LAL --as-of
    2026-10-19T23:00:00+00:00
```

Direct-file execution is also supported: `python Models/Game_Outcome/PPP/PPP.py ...` and `python Models/Game_Outcome/PPP/NBA_PPP.py ...`. Add `--refresh` only when public historical inputs should be refreshed. These commands do not call The Odds API. Without `--home` and `--away`, they prepare observations without generating a matchup. Use the explicit historical `--as-of` cutoff when reproducing an old estimate.

2. How the implementation maps to the PDFs

Reference	Code	Implemented behavior
PPP 2.1: consistent offensive/defensive PPP	`NflModelData`, `PPPModel._history`	Separates offensive plays/points from other scoreboard points and requires complete paired team-game rows.
PPP 2.2: multiplicative baseline	`matchup_ppp`	Returns `offensive_PPP * opposing_defensive_PPP / league_PPP`; exposed for explanation/tests.
PPP 2.3: jointly adjusted production	`PPPModel._fit`	The default live NFL model fits a regularized log-link offense/defense model with play exposure, home field and recency. It does not merely multiply raw team averages.
PPP 3.1-3.2: opportunities times efficiency	`PPPModel._means`	Shares a drive forecast between teams, matches each offense's plays per drive to the opposing defense, and adds separately estimated non-offensive points.
PPP 4.1: normal probability baseline	`PPPModel.predict`, `PPPForecast`	Learns bias and margin/total uncertainty from forward-held-out score errors.
PPP 4.2: possession/drive Monte Carlo	Not implemented	No clock/field-position transition simulator is claimed.
PPP 5.1: prices, pushes and expected value	Odds helpers, `MarketProbability`, `PPPForecast._settlement`	Implements implied probability, fair American price and conditional win probability excluding refunds. Expected-value staking and arbitrage execution are not implemented.
EPA 6.2-6.3: NBA adjusted PPP and shared pace	`NBAPPPModel._ratings`, `_means`	Uses the additive NBA baseline and estimated shared possessions.
EPA 7: total/margin covariance	Both `predict` methods	Uses opposite covariance signs for margin and total variances.

3. NFL data contract and scoring definitions

NflModelData joins public nflverse schedules/final scores, team box scores, nflfastR play-by-play and PFR player snap counts. It writes `data/modeling/nfl/team_games_<season>.csv` and source manifests. Raw files and generated forecasts stay under `data/modeling/nfl`.

Each model observation is one team in one completed regular/postseason game:

Field family	Fields used by NFL PPP	Units/meaning
Identity/time	`game_id`, `team`, `opponent`, `season`, `week`, `event_date`, `available_at`	Game IDs/teams are identifiers; the dates control historical availability.
Context	`is_home`, `neutral`	Indicators controlling the learned home effect.

Field family	Fields used by NFL PPP	Units/meaning
Volume	`offensive_plays`, `defensive_plays`, `offensive_drives`, `defensive_drives`	Actual prior-game counts, used only to learn future volume.
Offensive scoring	`offensive_points`, `offensive_points_allowed`	Points attributed to offensive possessions, not all scoreboard points.
Score target	`points`, `non_offensive_points`	Actual team final points, and the residual scoring component estimated separately.

Passes, sacks, runs, kneels and spikes count as offensive plays. Kicks, two-point tries, administrative records and no-play penalties do not count in that denominator. Offensive scoring includes offensive touchdowns, their linked conversions and made field goals. Defensive/special-teams touchdowns and their conversions, safeties and other residual points stay in the additional component. Thus a pick-six is not incorrectly treated as the offense's scoring efficiency.

The builder rejects duplicate game/play IDs, invalid possession ownership, missing drive IDs and disagreement between final PBP scores and the completed schedule. Team box scores remain actual team records. Maximum player snap counts are participation checks, not team play totals obtained by summing players. Provider EPA fields can be retained in the shared dataset, but PPP does not use EPA as its efficiency response.

`PPPModel._history` requires two rows per game, unique game/team identities, finite nonnegative count/scoring fields and positive play/drive exposure. It keeps only `available_at < cutoff` and games on earlier UTC calendar dates. The NFL builder conservatively sets availability to game date plus 36 hours. The shared service supplies a timezone-aware cutoff; the standalone NFL history method also accepts a naive timestamp by treating it as UTC.

4. NFL efficiency estimation

4.1 Intuitive baseline and paper example

For offense `i` facing defense `j`, define offensive points per play `O_i`, opposing offensive points allowed per play `D_j`, and league baseline `r_bar`. The paper's section 2.2 helper is:

```
matchup_PPP = O_i * D_j / r_bar
A vs B: 0.480 * 0.440 / 0.400 = 0.528
B vs A: 0.360 * 0.380 / 0.400 = 0.342
```

A larger defensive PPP allowed means a more permissive defense. This example verifies units and the helper, but raw rates still reflect schedule strength.

4.2 Default jointly fitted model

For prior team-game observation `g`, the log of expected offensive PPP is:

```
eta_g = alpha + home_coefficient * h_g + offense_effect[team]
              + defensive_vulnerability[opponent]
h_g   = (2 * is_home - 1) * (1 - neutral) / 2
expected_offensive_points_g = offensive_plays_g * exp(eta_g)
```

The home predictor is +0.5 for the home team, -0.5 for the away team and zero at neutral sites. This is the implemented context feature; weather, injuries and rest are possible PDF extensions but are not supplied here.

`_history` assigns recency weight `w_g = 2 ** (-age_days / 180)`. `_fit` builds a matrix with intercept, home indicator, offensive-team indicators and opposing defensive-team indicators, and minimizes:

```
sum_g w_g * [plays_g * exp(eta_g) - offensive_points_g * eta_g]
    + 0.5 * sum_k penalty_k * coefficient_k^2
```

This is a regularized Poisson **mean** objective with play exposure. It accepts zero-point games. It is not a claim that final-score variance is Poisson, and Poisson tail probabilities are not used to price spreads/totals.

The intercept starts at the log of weighted league points/plays. Newton steps solve the gradient/Hessian system with backtracking; team effects are centered and their removed means are absorbed into the intercept. The intercept has no penalty, team effects have penalty 20, and the home coefficient penalty is 2. Centering makes offense/defense effects interpretable relative to the intercept.

5. NFL volume and score means

`_means` selects each team's earlier observations and computes recency-weighted averages. With `avg(team, column)` denoting those averages:

```
N = mean(home offensive drives, home defensive drives,
         away offensive drives, away defensive drives)

L_team = 0.5 * [avg(team, offensive_plays) / avg(team, offensive_drives)
             + avg(opponent, defensive_plays) / avg(opponent, defensive_drives)]

expected_plays_team = N * L_team
mu_team = expected_plays_team * fitted_matchup_PPP + delta_team
```

Both teams share `N`, so their pace point estimates do not come from independent clocks. Plays per drive remain matchup-specific. This is a product-of-averages baseline, not a fitted stochastic model of drive length/clock covariance.

Let `league_extra` be the weighted league non-offensive points per team-game:

```
delta_team = [sum(w * team_non_offensive_points) + 2 * league_extra]
             / [sum(w) + 2]
```

The two units of prior weight shrink sparse team additional scoring toward the league. Score means are later adjusted by historical forecast bias. The paper's zero-extra worked example has `62 * 0.528 = 32.736` and `61 * 0.342 = 20.862`: total 53.598 and first-team margin 11.874. These example constants are not substituted for learned means in the app.

6. Chronological calibration and uncertainty

`PPPModel.predict` holds out entire `(season, week)` groups. A group's fit uses only observations available before its earliest game, reweights them at that historical cutoff, and predicts both teams without the held-out game's actual plays or scores. Too-small fits or team samples cannot produce forecasts.

For the latest 250 eligible held-out games, let error pair `e_g` be actual final scores minus predicted means. At least 40 error pairs are required. Their sample mean corrects forecast bias; their sample covariance `C` determines uncertainty:

```
mu_home, mu_away = fitted means + mean(error pairs)
mu_margin = mu_home - mu_away
mu_total  = mu_home + mu_away
variance_margin = C_home_home + C_away_away - 2 * C_home_away
variance_total  = C_home_home + C_away_away + 2 * C_home_away
```

Negative adjusted NFL score means are floored at zero. Nonpositive variance prevents a distribution estimate. Diagnostics preserve error pairs, covariance, training/validation counts, last training date and margin/total mean absolute errors. The normal margin/total curves are derived from these fitted quantities. The code does not generate joint score Monte Carlo samples.

These are forward-held-out errors used for **calibration fitting**. A separate, untouched future test set is still needed to establish predictive calibration or betting performance. Passing formula tests does not establish accuracy.

7. Settlement, market probabilities, fair odds and Edge

For American quote `a`, `implied_probability(a)` returns:

```
a > 0: 100 / (a + 100)
a < 0: abs(a) / (abs(a) + 100)
```

For example, -110 gives `110/210 = 52.380952%`. These are raw break-even probabilities containing the bookmaker's margin. The app does not remove vig.

`PPPForecast.spread(team, handicap)` prices `team_margin + handicap > 0`. For a home -3.5 bet, the modeled home margin must exceed 3.5. `_settlement` approximates an integer random variable by normal bins centered on integers. For threshold `t`, the boundary for the lower integer range is `ceil(t) - 1 + 0.5`; the upper range starts after `floor(t) + 0.5`. At integer thresholds, the gap is the push bin. For total markets, negative integer total bins are excluded at the lower boundary. `total(line, 'Under')` reverses the win/loss directions of the same total distribution.

The result is `MarketProbability(win, loss, push)`. The displayed probability is:

```
p_display = win / (win + loss)
fair American odds = +100 * (1-p_display) / p_display       if p_display <= 0.5
                   = -100 * p_display / (1-p_display)       if p_display > 0.5
```

For win 0.60, loss 0.35, push 0.05, the displayed probability is 63.1579% and the fair price is approximately -171.43. The push receives a refunded stake. Endpoint probabilities 0 and 1 have no finite fair quote and display missing. Hover values retain unconditional win/loss/push probabilities.

The dashboard computes `Edge = p_display - market_implied`, displayed in percentage points. For probability 0.80 and market -110, Edge is about +27.62 pp. This difference is a comparison statistic, not an arbitrage proof. Default highlight criteria require both probability >75% and Edge >5 pp. The shared criteria apply to both teams' alternate spreads and both Over/Under totals. Gold chart segments and markers identify the same qualifying quoted lines. Each market shows six selections initially; only Show more reveals the rest, and the bottom Show less control collapses them. Applying criteria does not automatically expand the market's extra selections.

The PDF's section 4.1 example uses sigma_margin=13 and sigma_total=13.5 to obtain an uncorrected win estimate about 81.95% and Over 51.5 about 56.17%. `normal_over` exposes that illustrative continuous calculation. Live estimates learn their own standard deviations and use settlement-aware pricing.

8. NBA-specific PPP implementation

8.1 Inputs and possession estimate

NbaModelData uses the existing NBA Stats/SportsDataverse historical adapters for team box scores. Regular season and postseason are separate inputs. It requires paired, consistent team/opponent, date and `AWAY @ HOME` matchup fields and finite nonnegative box-score counts. Duplicate team-game rows are rejected; incomplete/unplayed or inconsistent pairs are excluded. Tied full-game final scores are not valid NBA observations.

```
team_estimated_possessions = FGA + 0.44 * FTA - OREB + TOV
shared_game_possessions = mean(both teams' estimated possessions)
```

Offensive rebounds extend possessions instead of creating new ones. This is a box-score approximation; no exact event-state or EPV count is claimed. Actual overtime points and possessions remain included in full-game observations. `available_at` is conservatively the game's UTC date plus two days.

8.2 League rates, shrinkage and matchup

`NBAPPPModel._ratings` uses 90-day-half-life weights. League PPP is weighted points divided by weighted possessions; league pace is the weighted mean game possession count. For team shrinkage, weights are rescaled so the newest is 1, keeping the prior strength stable during an offseason.

Let `k=10`, league pace `P_bar`, league PPP `ell`, and `w` those rescaled weights:

```
offense_PPP = [sum(w * points) + k * P_bar * ell]
              / [sum(w * possessions) + k * P_bar]
defense_PPP = [sum(w * points_allowed) + k * P_bar * ell]
              / [sum(w * possessions) + k * P_bar]
team_pace = [sum(w * possessions) + k * P_bar] / [sum(w) + k]

matchup_PPP_A = offense_PPP_A + defense_PPP_B - ell
expected_possessions = (team_pace_A + team_pace_B) / 2
mu_A = expected_possessions * matchup_PPP_A
```

The additive helper follows EPA PDF section 6.2: `1.18 + 1.14 - 1.12 = 1.20`. With the separate section 6.3 example of 99 possessions and PPP values 1.18/1.12, score means are 116.82/110.88, total 227.70 and first-team margin 5.94. These are teaching examples. The operational model learns actual inputs.

The current NBA efficiency ratings are this shrunk additive baseline, not a joint schedule-strength/lineup model. Nonpositive matchup efficiency fails. Home advantage enters through learned home/away calibration bias rather than an explicit rating predictor. Neutral games use their common bias instead of the home differential.

8.3 NBA validation and spread settlement

The NBA model holds out entire dates and refits using only earlier available observations. It requires 100 training games, 10 prior appearances per selected team and 100 validation games; it uses the latest 300 validation pairs for bias and covariance. Totals/margin variances use the formulas in section 6 above.

`NBAPPPForecast.spread` removes the normal distribution's zero-margin bin and renormalizes other outcomes because full-game NBA games cannot end tied after overtime. Pick'em has zero push mass; other integer handicaps retain their push bins. This is an approximation, not an explicit overtime simulator. The normal total curve remains a marginal approximation; the class is not a fully coherent discrete joint score simulation conditioned on every basketball scoring rule.

NBA EPA/EPV is available as an independently fitted event-based research baseline; see the EPA documentation. It uses reconstructed public events and its own calibration, rather than relabeling these PPP forecasts.

9. Configuration and saved-market integration

Setting	NFL default	NBA default
Recency half-life	180 days	90 days
Efficiency regularization	Ridge 20; home 2	10 equivalent-game prior
Minimum training games	50	100
Minimum appearances per selected team	4	10
Minimum validation games	40	100
Maximum calibration pairs	250	300
Input availability lag	Game date +36 hours	UTC game date +2 days

These are explicit initial settings, not tuned or production-certified values.

PPPMarketService and NBAPPPMarketService connect saved quotes to models. They:

1. Resolve recognized teams and set cutoff to the earlier of snapshot time and kickoff/tipoff. Post-start snapshots are labeled as pregame comparisons.
2. Hash model version, teams, cutoff, full configuration and neutral status.
3. Reuse memory or `data/modeling/<sport>/forecasts/ppp_<hash>.json` if present.
4. Otherwise prepare the previous/current sport seasons, fit and save the forecast, source notices, input-file hashes and validation diagnostics.

NFL season selection rolls over in March; NBA uses October. Historical source files can include later corrections, so time filtering prevents future-game leakage but does not reconstruct the exact vintage of every old statistic. Standalone CLI matchup exports use readable `ppp_<home>_<away>.json` names; they are not the hashed live-service cache files.

Opening saved odds, applying criteria and drawing charts never pull paid odds. Public historical downloads can occur on the first uncached forecast. The UI shows model probability, fair odds and Edge for spread/total lines, with separate home, away, Over and Under charts. Player props have no PPP probability model.

10. Verification and practical limits

Run `python -m unittest discover -s tests -q`. The existing PPP, NBA market and EPA tests verify paper examples, odds conversions, pushes, cutoff filtering, chronological behavior, model selection, cache reuse and dashboard callbacks. Tests consume no paid API credits. App/formula checks and historical calibration diagnostics do not prove future forecasting accuracy.

The PDFs' preferred drive/possession simulators, football key-number models, injury/weather/rotation updates, sport-specific overtime transitions, staking, cross-book arbitrage, and NCAAF calibration are not implemented here. Football key numbers are only normal integer bins; NBA overtime is represented by past full-game observations and the no-tie spread adjustment. None of the outputs should be described as the full advanced system proposed in either PDF.

Build provenance

The following SHA-256 hashes identify the reviewed calculation/service files. They are documentation provenance, not a runtime cache identity.

Models/Game_Outcome/PPP/README.md
`eee05fe3e21de64d27b24fbb48cbc734c9bf866fa78306c504f52fe38b415f9f`

Models/Game_Outcome/PPP/PPP.py
`39bdd558188e25b0e55b27569ddeae9c3515a3985819ba7e314e9c21bd5f23a8`

Models/Game_Outcome/PPP/NBA_PPP.py
`4cb7977f1273059fc96af0cbdeb8985ab5156724711ebc6360682f3655574c1c`

sports_analytics/ppp_service.py
`f48d098f2ab4ff440acc748e214e191a753f6d0c03b9d2f31db202bcdd984f68`

sports_analytics/nba_ppp_service.py
`2b79f762beca1e015e793f9e52bfba6e112c0f1a59a5436c50279c06f655cbe8`

sports_analytics/nfl_model_data.py
`4cb555e0319312f841bf0ceb6392c7ee6f3a7c7d10a5899cb9fb6c2230597937`

sports_analytics/nba_model_data.py
`9dc05d6234ed4c84e3603607a3fae3905c7776eaf6a577a8a92e50ee126b38b6`